

Furnish Fusion: Your Online Destination for Stylish Living

Project Description:

Furnish Fusion – Your Online Destination for Stylish Living is a cloud-based furniture e-commerce platform developed using Flask with a modular route-based architecture for admin, user, product, order, and budget planning functionalities. Hosted on AWS EC2 for reliable deployment and scalability, the application integrates DynamoDB for dynamic data storage and management of users, products, carts, wishlists, and orders. The platform provides secure user registration and login, product browsing, cart and wishlist management, checkout processing, order tracking, and a dedicated budget planner feature to help users plan their purchases efficiently. An admin dashboard enables product management, order monitoring, and customer interaction, ensuring smooth platform operations. AWS SNS is used to send real-time email notifications for registrations and order confirmations, while IAM roles ensure secure and controlled access to AWS services. Overall, Furnish Fusion delivers a secure, scalable, and feature-rich online shopping experience tailored for modern home furnishing needs.

Scenario 1: Seamless Online Furniture Shopping Experience

In the Furnish Fusion platform, AWS EC2 provides a reliable and scalable infrastructure that allows multiple users to browse and shop simultaneously without performance issues. For example, a customer can log in, explore different furniture categories such as sofas, beds, or dining sets, and view detailed product descriptions and pricing. They can add items to their cart or wishlist and proceed to checkout smoothly. Flask handles backend operations such as user authentication, cart updates, and order processing, ensuring fast response times even during peak shopping periods.

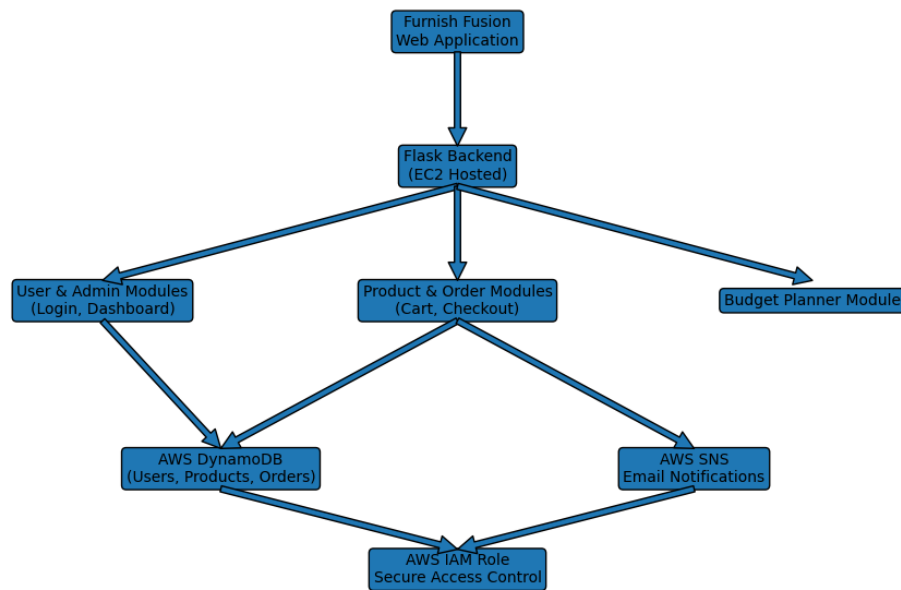
Scenario 2: Secure Checkout, Order Tracking, and Real-Time Notifications

When a customer proceeds to checkout on Furnish Fusion, the system securely processes the order details, including selected products, quantities, pricing, and user information. Flask manages the backend transaction logic while DynamoDB stores the order records instantly for future tracking and management. Once the order is successfully placed, AWS SNS triggers an automatic email notification containing the complete purchase summary, order ID, and estimated delivery details. Users can later log in to view their order history and track their purchases, while admins can monitor and manage orders through the dashboard. This end-to-end workflow ensures transparency, reliability, and a smooth post-purchase experience for customers.

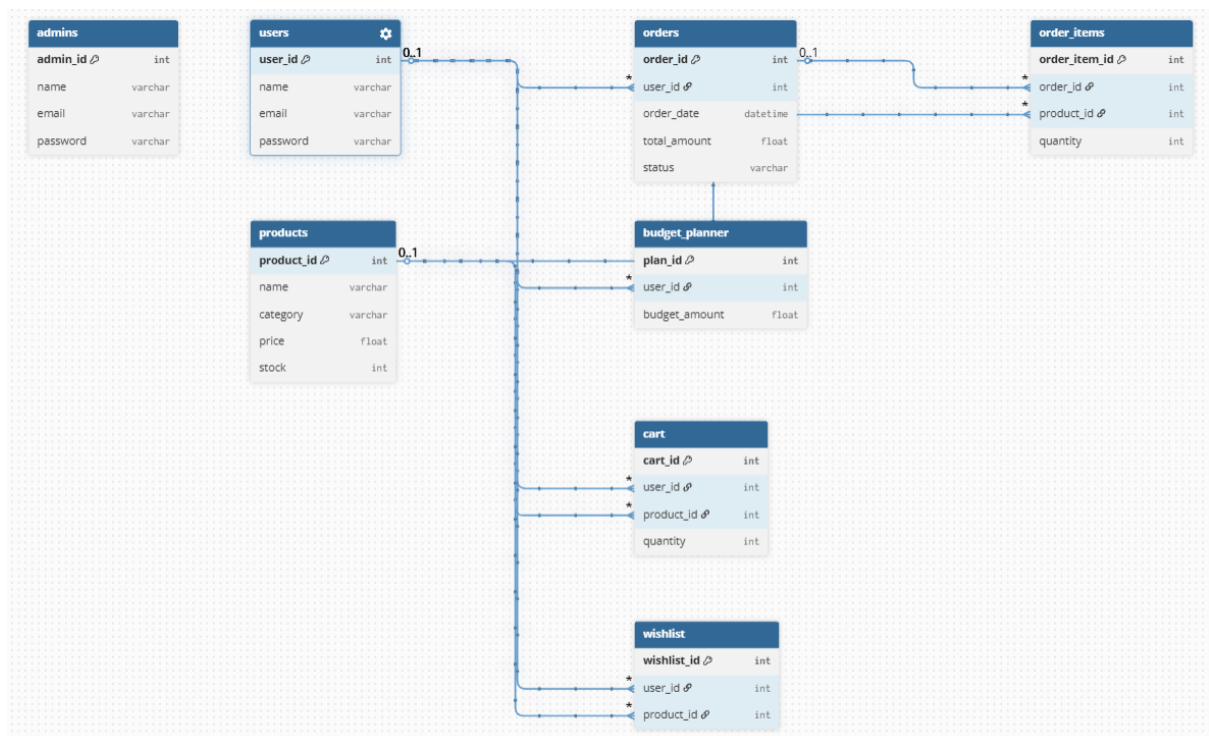
Scenario 3: Admin Management and Product Control

Furnish Fusion provides an admin dashboard that allows administrators to add, update, or remove products, manage customer orders, and monitor overall platform activity. Through dedicated admin routes, the system ensures controlled access using IAM roles for secure AWS service interaction. For example, an admin can log in to add a new furniture collection, update pricing, or view recent orders placed by customers. This structured management system ensures smooth platform operations and maintains accurate, up-to-date product listings for users.

AWS Architecture :



Entity Relationship (ER) Diagram :



Pre-requisites:

1. AWS Account Setup: AWS Account Setup
2. Understanding IAM: IAM Overview
3. Amazon EC2 Basics: EC2 Tutorial
4. DynamoDB Basics: DynamoDB Introduction
5. SNS Overview: SNS Documentation
6. Git Version Control: Git Documentation

Project Workflow :

1. EC2 Instance Setup

Activity 6.1: Launch an EC2 instance to host the Flask application.

Activity 6.2: Configure security groups for HTTP, and SSH access

2. SNS Notification Setup

Activity 3.1: Create SNS topics for Ordering Products notifications.

Activity 3.2: Subscribe users to SNS email notifications.

3. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for User Data and Order Products.

4. IAM Role Setup

Activity 5.1: Create IAM Role

Activity 5.2: Attach Policies

5. Deployment on EC2

Activity 7.1: Upload Flask Files

Activity 7.2: Run the Flask App

6. Backend Development and Application Setup

Activity 4.1: Develop the Backend Using Flask.

Activity 4.2: Integrate AWS Services Using boto3.

7. Testing and Deployment

Activity 8.1: Conduct functional testing to verify user registration, login, Order requests, and notifications.

Milestone 1 : EC2 Instance Setup

Activity 1.1 : Taking a Amazon Linux server and naming (FurnishFusionApplication)

Name and tags [Info](#)

Name
FurnishFusionApplication [Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents **Quick Start**

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux Debian [Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

▼ Summary

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.10...[read more](#)
ami-0532be01f26a3de55

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Activity 2.2 : Taking an Instance type as t3.micro and creating new Keypair (TrovenLabKP)

▼ Instance type [Info](#) [Get advice](#)

Instance type
t3.micro Family: t3 2 vCPU 1 GiB Memory Current generation: true Free tier eligible [Compare instance types](#) ☐ All generations

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*
TrovenLabKP [Create new key pair](#)

▼ Network settings [Info](#) [Edit](#)

▼ Summary

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.10...[read more](#)
ami-0532be01f26a3de55

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Activity 1.3 : As default SSH is allowed so adding new rules for Custom TCP with port 5000 and HTTP port on 80.

▼ Security group rule 2 (TCP, 5000, 0.0.0.0/0)

Remove

Type | Info

Custom TCP ▼

Protocol | Info

TCP

Port range | Info

5000

Source type | Info

Custom ▼

Source | Info

🔍 Add CIDR, prefix list or security group

0.0.0.0/0 ✕

Description - optional | Info

e.g. SSH for admin desktop

▼ Security group rule 3 (TCP, 80, 0.0.0.0/0)

Remove

Type | Info

HTTP ▼

Protocol | Info

TCP

Port range | Info

80

Source type | Info

Custom ▼

Source | Info

🔍 Add CIDR, prefix list or security group

0.0.0.0/0 ✕

Description - optional | Info

e.g. SSH for admin desktop

Activity 1.4 : So instance is in working state

[Alt+S]

United States (N. Virginia)
Account ID: 7138-8179-4827
rsocaccount-new/6837fdd8dc145df1d9fb359f

EC2 > Instances

Instances (1) [Info](#)

Last updated 2 minutes ago

[Connect](#)
[Instance state](#)
[Actions](#)
[Launch instances](#)

All states

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	FurnishFusionApplication	i-03ddb34477f0c59f	Running	t3.micro	Initializing	View alarms +	us-east-1f	ec2-34-236-134-29.

Milestone 2: SNS Notification Setup

Naming the Project Name :

Application Integration

Amazon Simple Notification Service

Pub/sub messaging for microservices and serverless applications.

Amazon SNS is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and event-driven serverless applications. Amazon SNS provides topics for high-throughput, push-based, fire-and-forget messaging. Amazon SNS also provides message queues for guaranteed, ordered, and idempotent messaging.

Create topic

Topic name

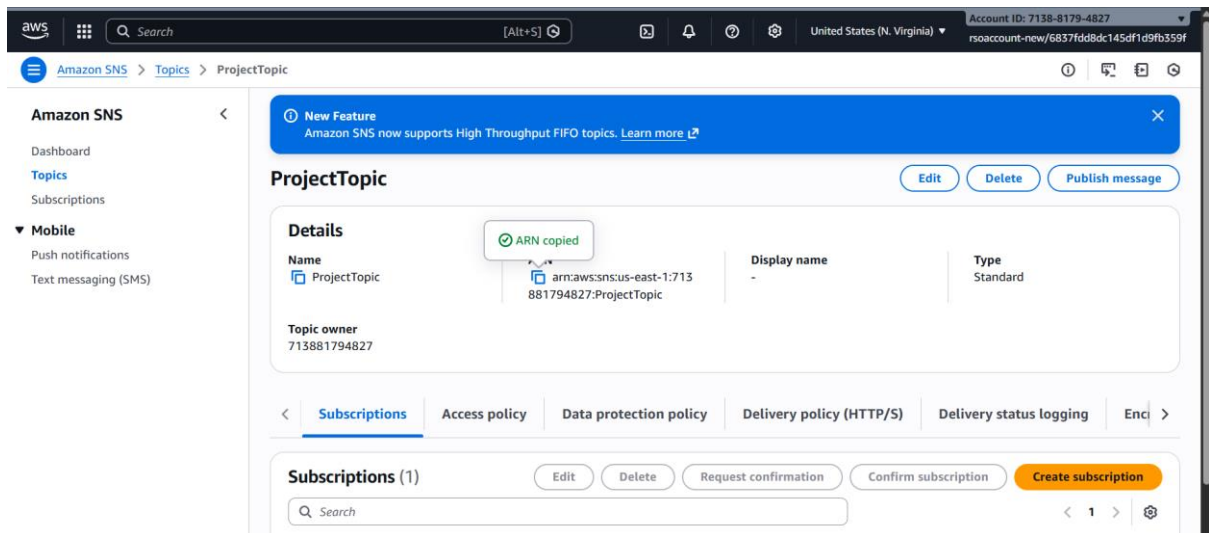
A topic is a message channel. When you publish a message to a topic, it fans out the message to all subscribed endpoints.

ProjectTopic

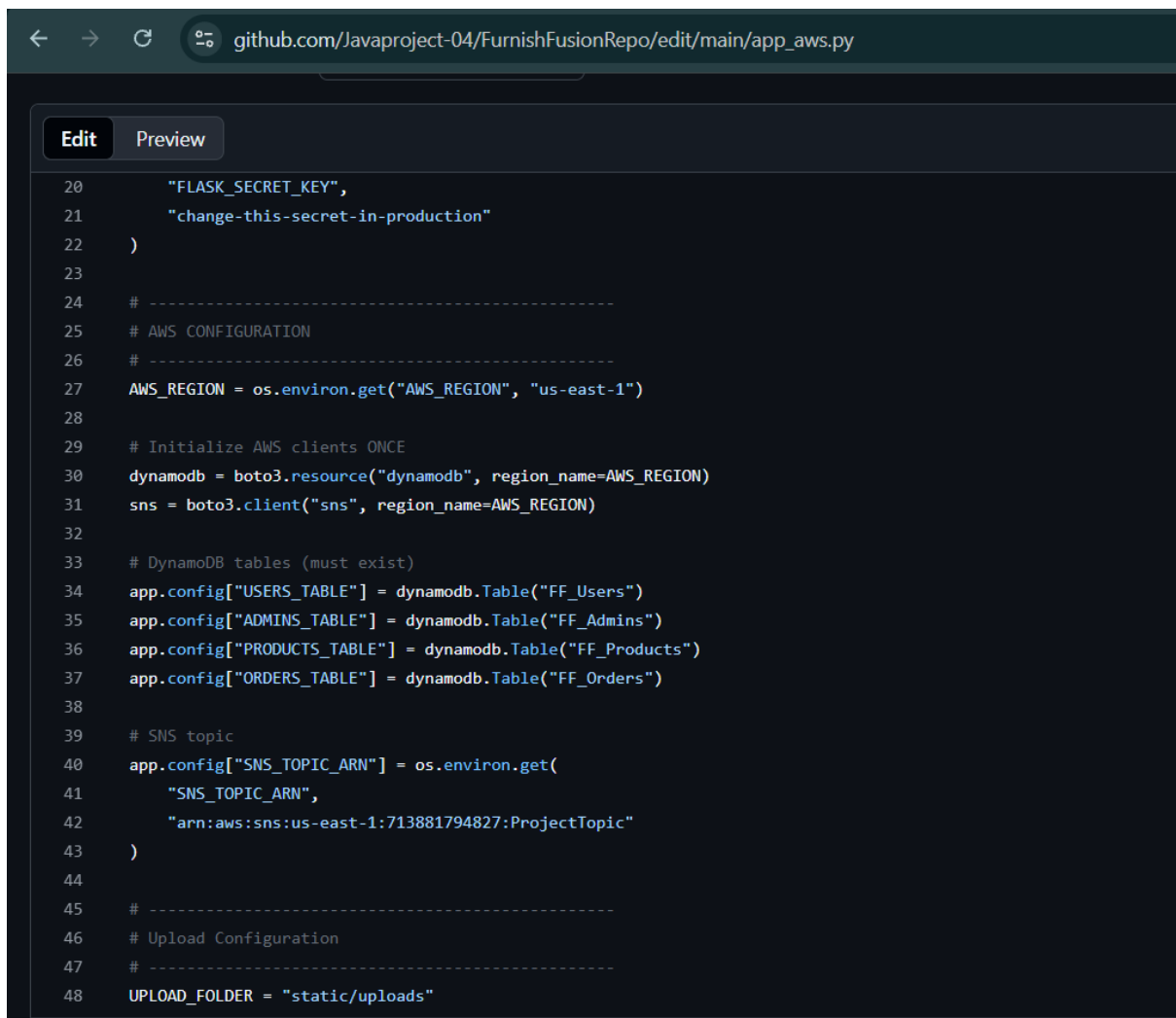
Next step

[Start with an overview](#)

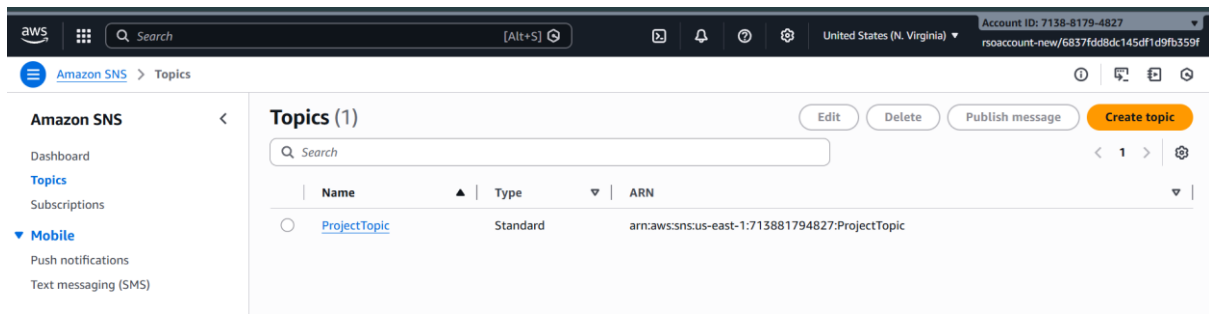
Project Topic created and also copy the ARN :



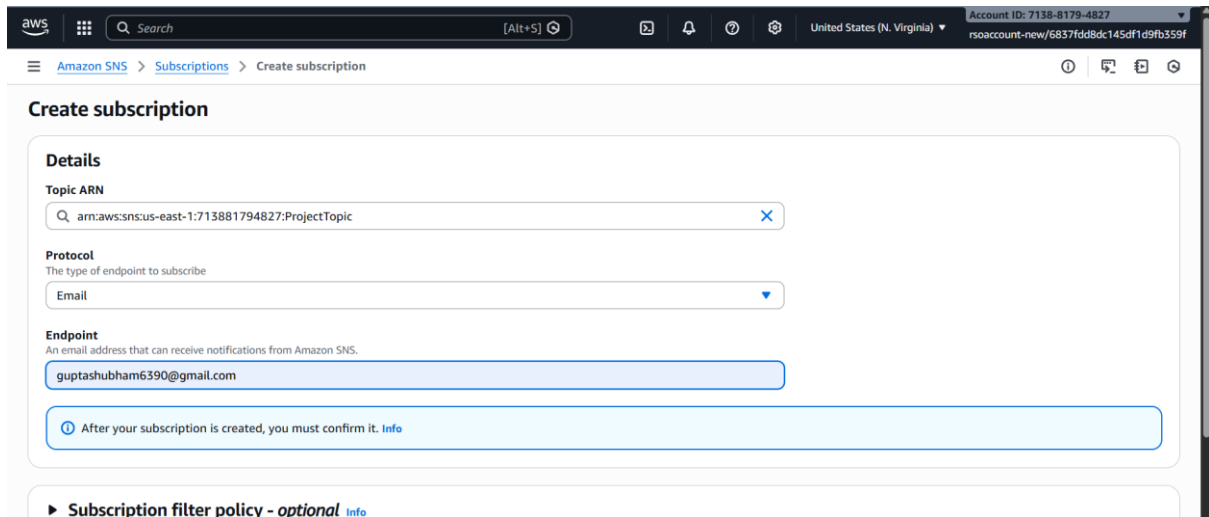
Paste the ARN to app_aws.py code where arn require for project :



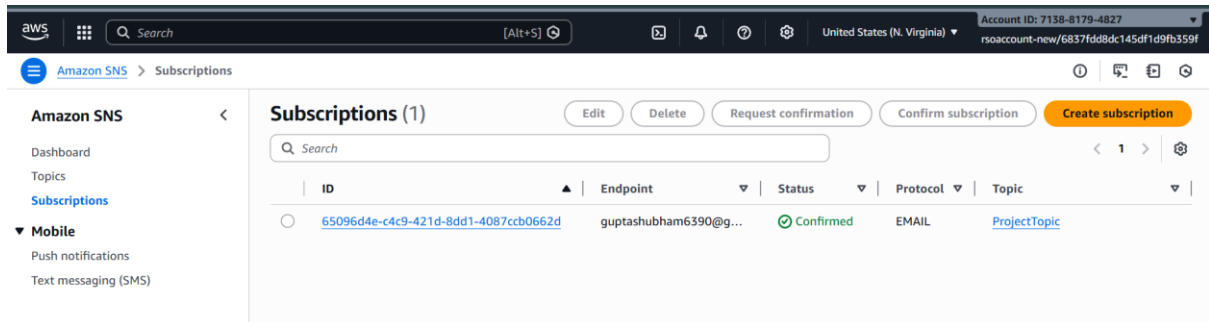
Now project topic is successfully associated with project :



Now making the Subscription of project by adding Mail :

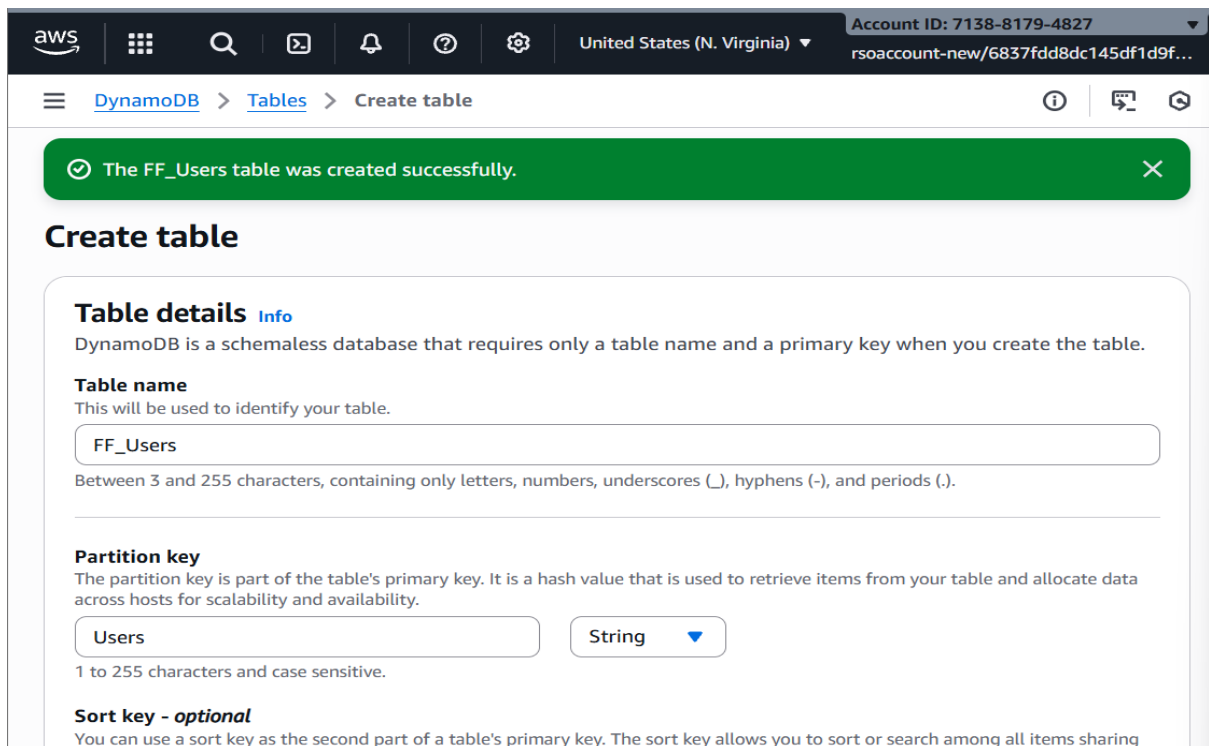


Now created the Subscription to a ProjectTopic:



Milestone 3: MongoDBDatabase Creation and Setup :

Creating the table named FF_Users for users data store :



The screenshot shows the AWS Management Console interface for creating a DynamoDB table. At the top, a green notification bar states: "The FF_Users table was created successfully." Below this, the "Create table" page is displayed. Under the "Table details" section, the "Table name" is "FF_Users". The "Partition key" is "Users" with a data type of "String". The "Sort key - optional" field is empty. The console header shows the account ID "7138-8179-4827" and the region "United States (N. Virginia)".

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

FF_Users

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

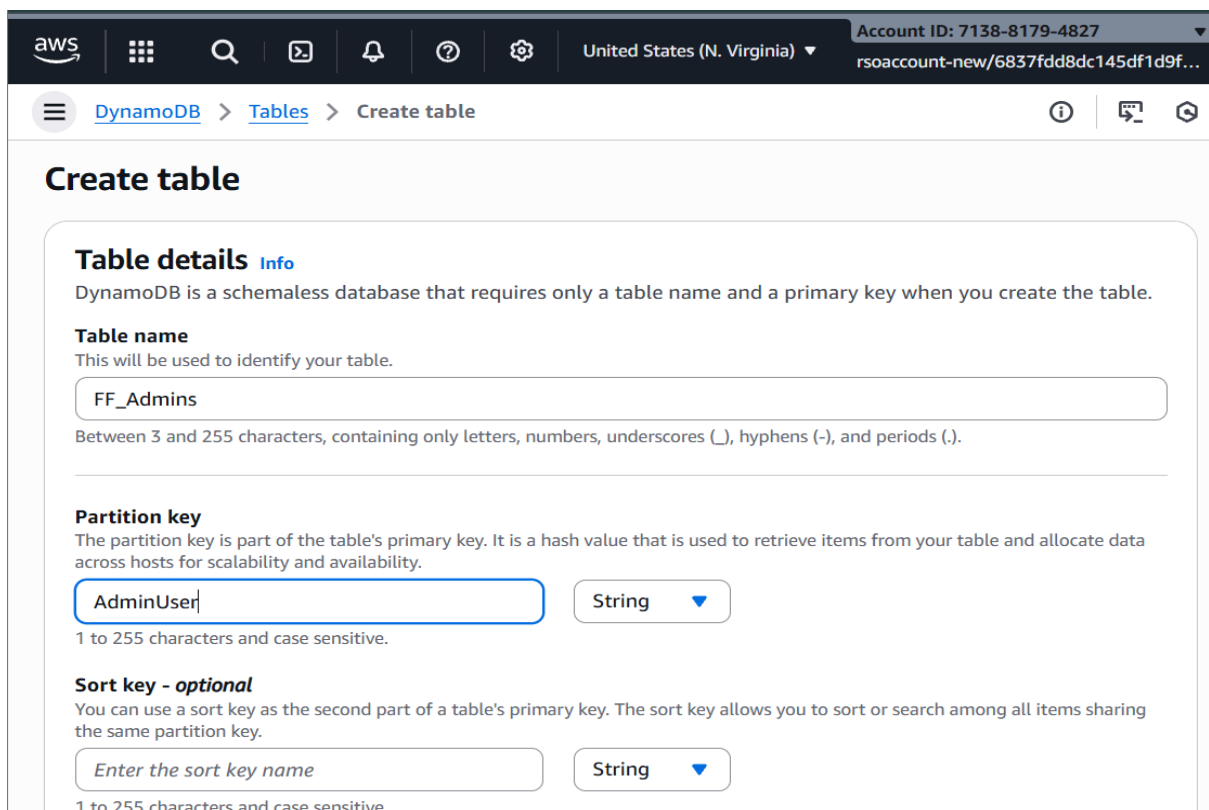
Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

Users String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing

Creating the table named FF_Admin for Admin data store :



The screenshot shows the AWS Management Console interface for creating a DynamoDB table. The "Table name" is "FF_Admins". The "Partition key" is "AdminUser" with a data type of "String". The "Sort key - optional" field is empty with the placeholder text "Enter the sort key name". The console header shows the account ID "7138-8179-4827" and the region "United States (N. Virginia)".

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

FF_Admins

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

AdminUser String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

1 to 255 characters and case sensitive.

Creating the table named FF_Products for products data store :

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

FF_Products

Between 3 and 255 characters, containing only letters, numbers, underscores (`_`), hyphens (`-`), and periods (`.`).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

ProductName String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Creating the table named FF_Orders for orders data store :

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

FF_Orders

Between 3 and 255 characters, containing only letters, numbers, underscores (`_`), hyphens (`-`), and periods (`.`).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

Order_id String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

All this are the Tables :

Tables (4) [Info](#) Last updated February 6, 2026, 19:25 (UTC+5:30) [Actions](#) [Delete](#) [Create table](#)

Filter by tag: Any tag key Filter by tag value: Any tag value < 1 > [Settings](#)

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions
☐	FF_Admins	Active	AdminUser (S)	-	0	0
☐	FF_Orders	Active	Order_id (S)	-	0	0
☐	FF_Products	Active	ProductName (S)	-	0	0
☐	FF_Users	Active	Users (S)	-	0	0

Milestone 4: IAM Role Setup

IAM role for aws services :

Step 1: Select trusted entity [Info](#)

Step 2: Add permissions
Step 3: Name, review, and create

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

As we are using IAM role for EC2 :

Step 3: Name, review, and create

AWS service
Allow AWS services like EC2, Lambda, or others to perform actions in this account.

Web identity
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

SAML 2.0 federation
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

Custom trust policy
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
EC2

Choose a use case for the specified service.

Use case
EC2

Naming the Role Name as StudentUser :

Step 1: Select trusted entity
Step 2: Add permissions
Step 3: **Name, review, and create**

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.
StudentUser
Maximum 64 characters. Use alphanumeric and '+', '=', '@', '-', '_' characters.

Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.
Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\[\]]#\$%^&*(){};:'"~`

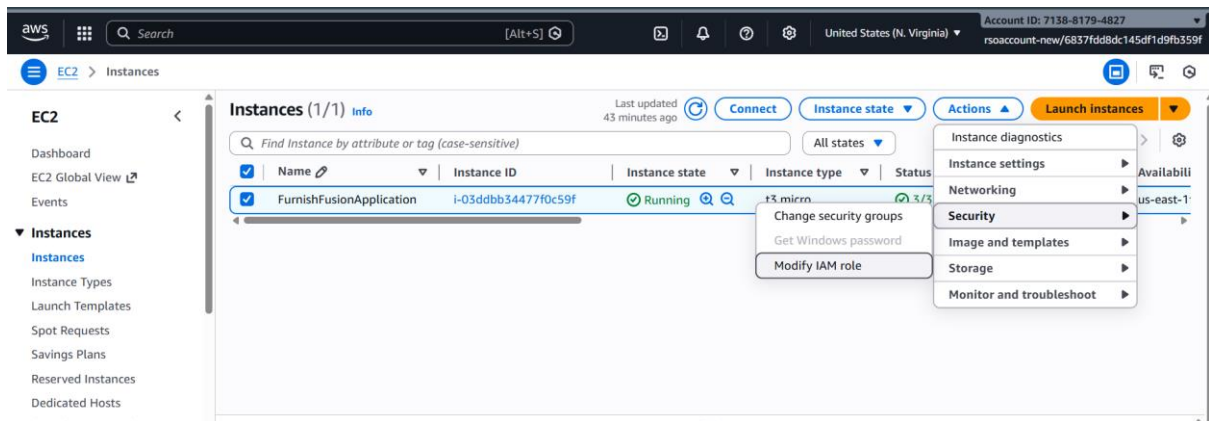
Adding different permissions like (DynamoDB full access, SNS full access, EC2 full access) :

Step 2: Add permissions

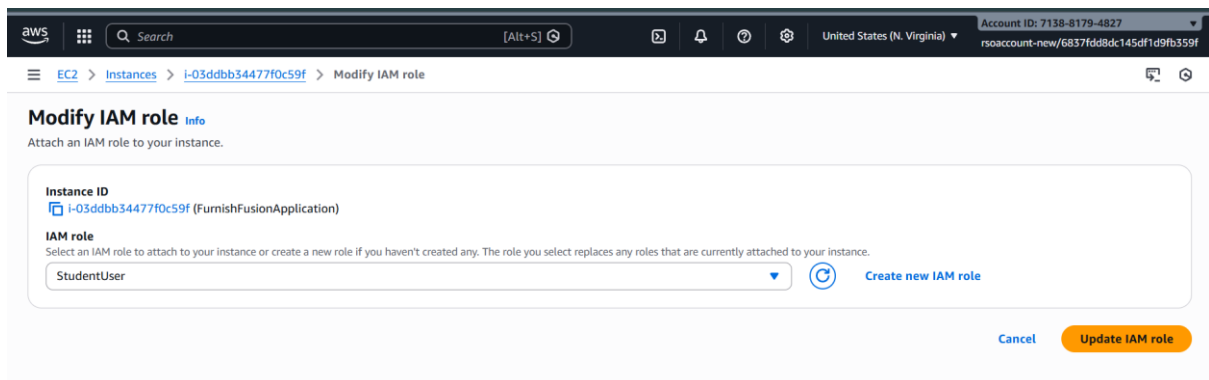
Permissions policy summary

Policy name	Type	Attached as
AmazonDynamoDBFullAccess	AWS managed	Permissions policy
AmazonEC2FullAccess	AWS managed	Permissions policy
AmazonSNSFullAccess	AWS managed	Permissions policy

Giving EC2 instance a permission for IAM roles to this EC2 instance :

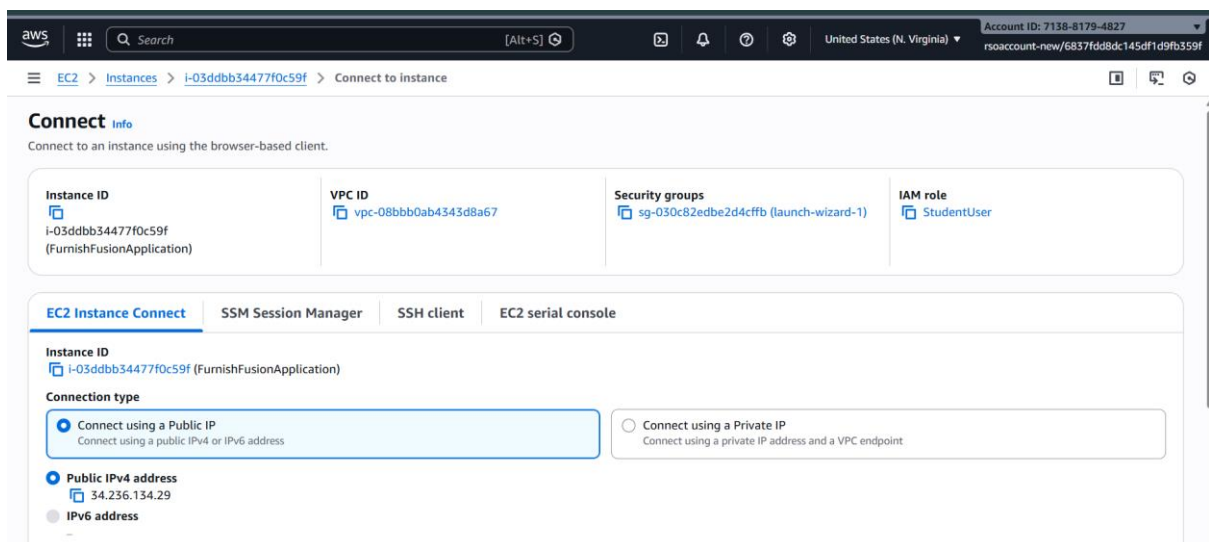


Attaching IAM role (StudentUser) to EC2 instance :



Milestone 5: Deployment on EC2

Now connecting to EC2 Amazon Linux Terminal via Ec2 instance connect :



A screenshot of the AWS Management Console interface. The top navigation bar shows the AWS logo, a grid icon, a search icon, a document icon, a notification bell, a help icon, and a settings gear. The region is set to 'United States (N. Virginia)'. The account ID is '7138-8179-4827'. The breadcrumb trail shows 'rsoaccount-new/6837fdd8dc145df1d9f...'. The main content area displays the Amazon Linux 2023 logo, which is a stylized tree-like structure made of hash symbols and backslashes. To the right of the logo, the text 'Amazon Linux 2023' is displayed. Below the logo, the URL 'https://aws.amazon.com/linux/amazon-linux-2023' is shown. At the bottom of the screenshot, a terminal window is visible with the command '[ec2-user@ip-172-31-73-63 ~]\$ sudo yum update -y'.

```
[fec2-user@ip-172-31-73-63 ~]$ git clone https://github.com/Javaproject-04/FurnishFusionRepo.git
Cloning into 'FurnishFusionRepo'...
remote: Enumerating objects: 84, done.
remote: Counting objects: 100% (84/84), done.
remote: Compressing objects: 100% (66/66), done.
remote: Total 84 (delta 19), reused 73 (delta 14), pack-reused 0 (from 0)
Receiving objects: 100% (84/84), 963.44 KiB | 50.71 MiB/s, done.
Resolving deltas: 100% (19/19), done.
[fec2-user@ip-172-31-73-63 ~]$
```

```

Receiving default: 100% (19/197) done.
[ec2-user@ip-172-31-73-63 ~]$ ls
FurnishFusionRepo
[ec2-user@ip-172-31-73-63 ~]$ cd FurnishFusionRepo
[ec2-user@ip-172-31-73-63 FurnishFusionRepo]$ ls
Dockerfile  api_aws.py  budget_planner.py  models.sql      static          utils.py
README.md   app.py      db.py              requirements.txt templates        view_db.py
__pycache__ app_aws.py  furnishfusion.db   routes          test_app_aws.py
[ec2-user@ip-172-31-73-63 FurnishFusionRepo]$

```

```
[ec2-user@ip-172-31-73-63 FurnishFusionRepo]$ pip3 install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting Flask
  Downloading flask-3.1.2-py3-none-any.whl (103 kB)
    |████████████████████████████████████████| 103 kB 17.3 MB/s
Collecting boto3
  Downloading boto3-1.42.43-py3-none-any.whl (140 kB)
    |████████████████████████████████████████| 140 kB 36.1 MB/s
Collecting moto[Server]
  Downloading moto-5.1.20-py3-none-any.whl (6.4 MB)
    |████████████████████████████████████████| 6.4 MB 108.0 MB/s
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
    |████████████████████████████████████████| 225 kB 115.5 MB/s
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting importlib-metadata>=3.6.0
```

Now running the app_aws.py file to run the application :

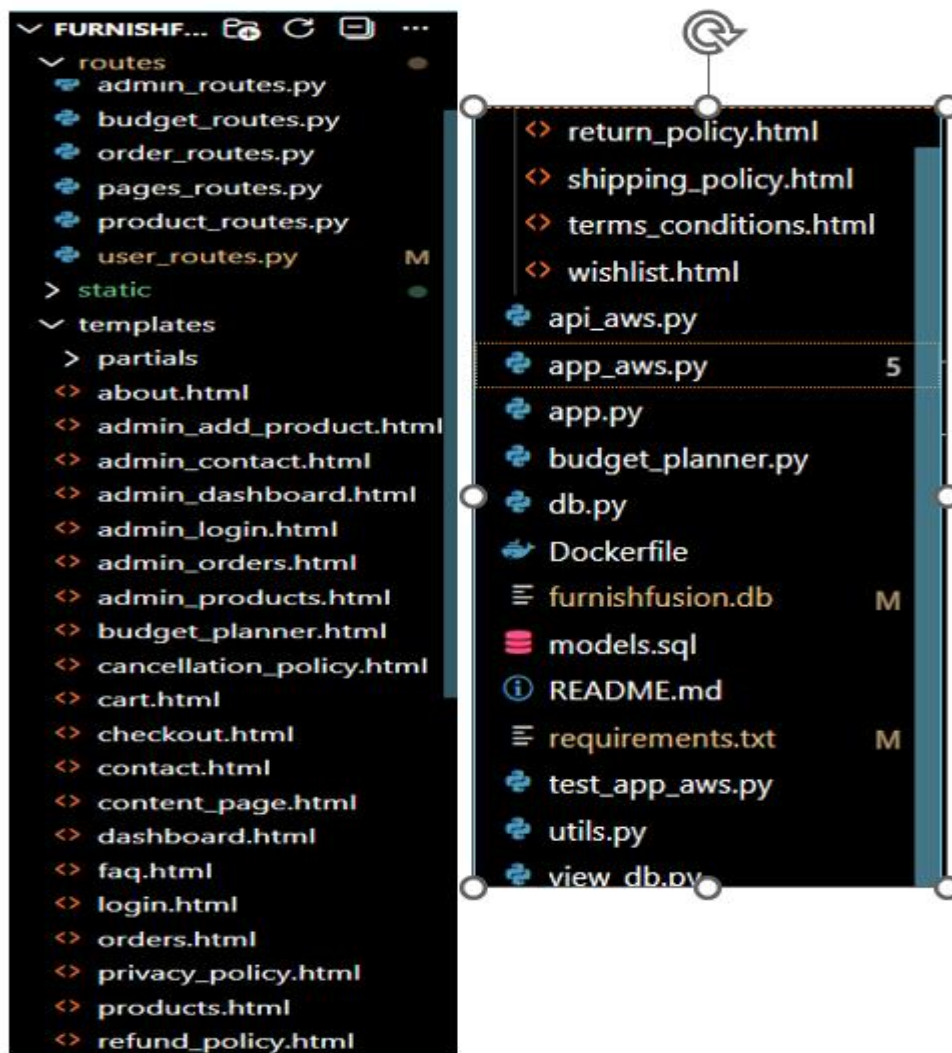
```
[ec2-user@ip-172-31-73-63 FurnishFusionRepo]$ python3 app_aws.py
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
* Serving Flask app 'app_aws'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.73.63:5000
Press CTRL+C to quit
```

i-03ddbb34477f0c59f (FurnishFusionApplication)



PublicIPs: 34.236.134.29 PrivateIPs: 172.31.73.63

Milestone 6: Backend Development and Application Setup



aws_app.py code :

OPEN EDITORS	app_aws.py > ...
FURNISHFUSION templates checkout.html contact.html content_page.html dashboard.html faq.html login.html orders.html privacy_policy.html products.html refund_policy.html register.html return_policy.html shipping_policy.html terms_conditions.html wishlist.html api_aws.py app_aws.py 5 app.py budget_planner.py db.py Dockerfile furnishfusion.db M models.sql README.md requirements.txt M test_app_aws.py utils.py view_db.py	<pre> 1 from flask import Flask 2 from db import init_db, close_db 3 from routes.user_routes import user_bp 4 from routes.product_routes import product_bp 5 from routes.order_routes import order_bp 6 from routes.admin_routes import admin_bp 7 from routes.budget_routes import budget_bp 8 from routes.pages_routes import pages_bp 9 import os 10 import boto3 11 from botocore.exceptions import ClientError 12 13 # ----- 14 # Flask App Setup 15 # ----- 16 app = Flask(__name__) 17 18 # Secret key (AWS safe) 19 app.secret_key = os.environ.get(20 "FLASK_SECRET_KEY", 21 "change-this-secret-in-production" 22) 23 24 # ----- 25 # AWS CONFIGURATION 26 # ----- 27 AWS_REGION = os.environ.get("AWS_REGION", "us-east-1") 28 29 # Initialize AWS clients ONCE 30 dynamodb = boto3.resource("dynamodb", region_name=AWS_REGION) 31 sns = boto3.client("sns", region_name=AWS_REGION) 32 33 # DynamoDB tables (must exist) 34 app.config["USERS_TABLE"] = dynamodb.Table("FF_Users") 35 app.config["ADMINS_TABLE"] = dynamodb.Table("FF_Admins") 36 app.config["PRODUCTS_TABLE"] = dynamodb.Table("FF_Products") 37 app.config["ORDERS_TABLE"] = dynamodb.Table("FF_Orders") </pre>
TIMELINE	

Milestone 7: Testing and Deployment

Project Running on <http://34.236.134.29/5000>

Users Login Page :

The screenshot shows a web browser window with the URL `34.236.134.29:5000/login`. The page has a dark blue header with navigation links: [Home](#), [About](#), [Products](#), [Cart](#), [Dashboard](#), [Orders](#), and [Logout](#). The main content area is a light blue box with a white background. It features a large heading "Welcome Back" and a subheading "Login to FurnishFusion". Below this are two buttons: "User Login" (blue) and "Admin Login" (grey). The "User Login" button is selected. Below the buttons are two input fields: "Email Address" (containing `guptashubham6390@gmail.com`) and "Password" (containing masked characters). A "Login" button is below the password field. At the bottom, there is a link "Don't have an account? Register here".

Not secure 34.236.134.29:5000/login

Welcome Back

Login to FurnishFusion

User Login Admin Login

Email Address

guptashubham6390@gmail.com

Password

.....

Login

or

Don't have an account? [Register here](#)

Main Dashboard :

The screenshot shows a web browser window with the URL `34.236.134.29:5000/dashboard`. The page has a dark blue header with navigation links: [Home](#), [About](#), [Products](#), [Cart](#), [Dashboard](#), [Orders](#), and [Logout](#). The main content area is a light blue box with a white background. It features a large heading "Welcome to FurnishFusion" and a subheading "Your personalized dashboard". Below this are three cards: "TOTAL ORDERS" (1), "TOTAL SPENT" (₹35000.00), and "MEMBER SINCE" (2026-01-16). At the bottom, there is a "Quick Links" section with three buttons: "Browse Products", "View Cart", and "All Orders".

Not secure 34.236.134.29:5000/dashboard

FurnishFusion

About Products Cart Dashboard Orders Logout

Welcome back, Shubham Gupta!

Welcome to FurnishFusion

Your personalized dashboard

Budget Planner

Shubham Gupta
guptashubham6390@gmail.com
Logout

TOTAL ORDERS
1

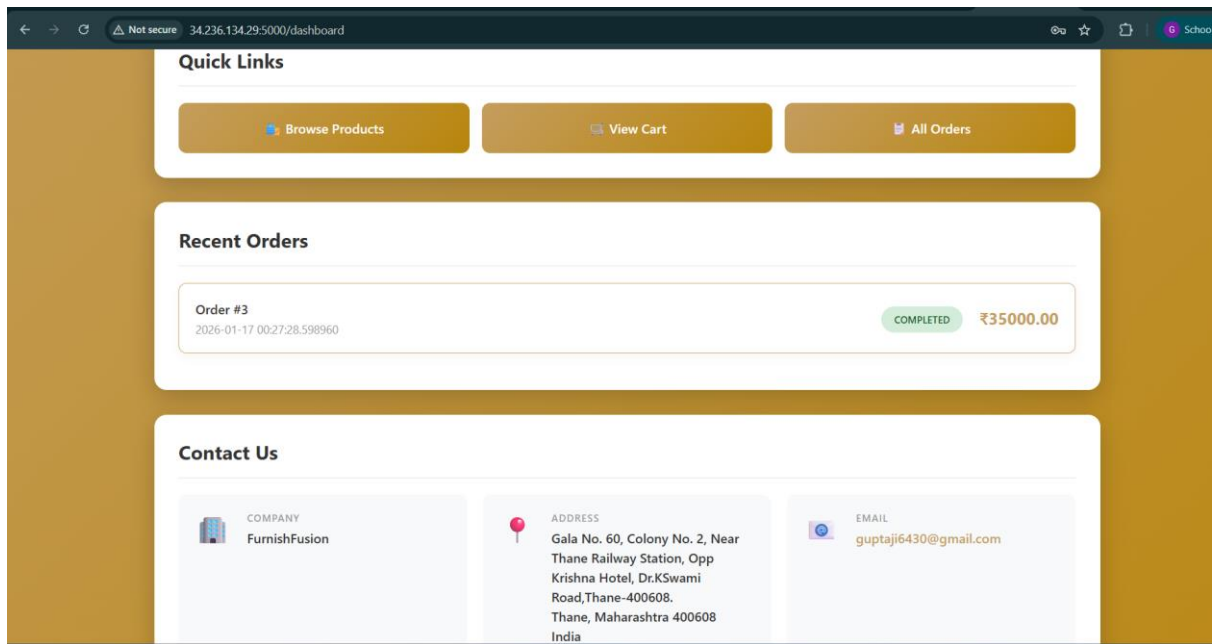
TOTAL SPENT
₹35000.00

MEMBER SINCE
2026-01-16

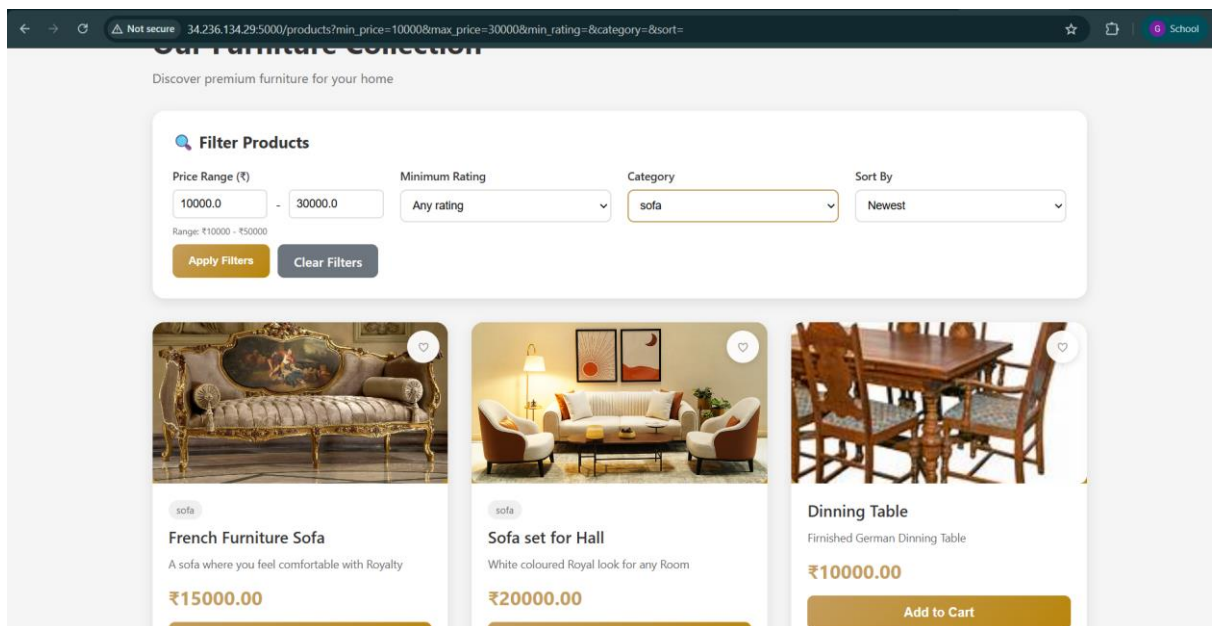
Quick Links

Browse Products View Cart All Orders

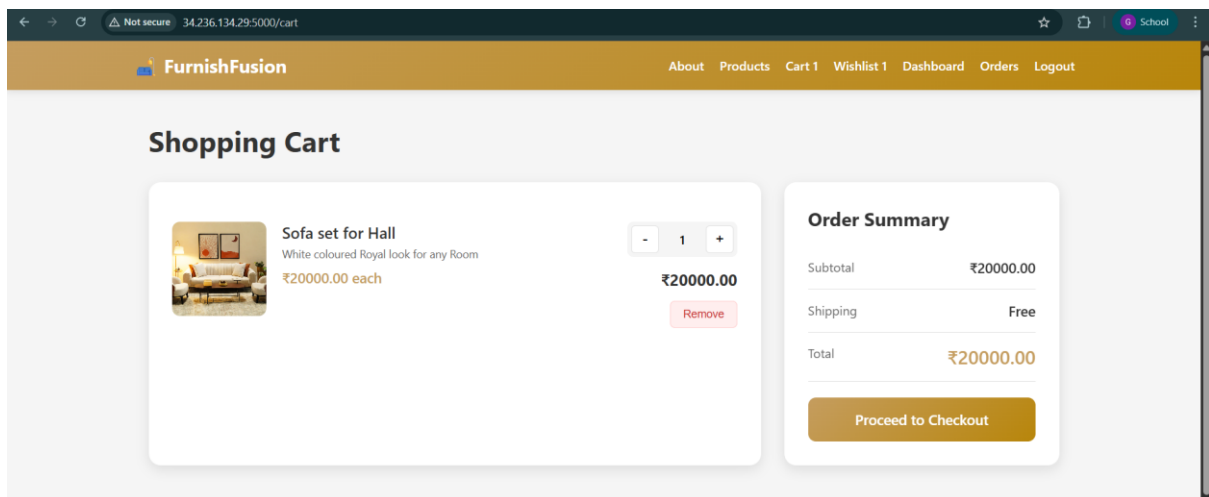
UserSide Dashboard can see Recent Orders :



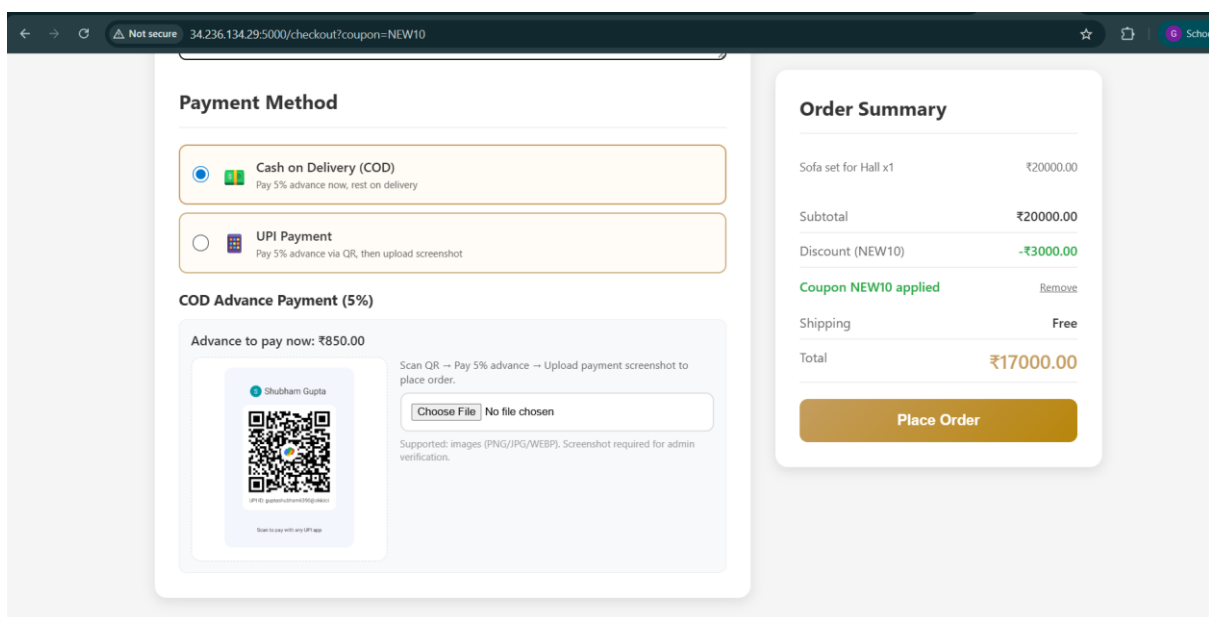
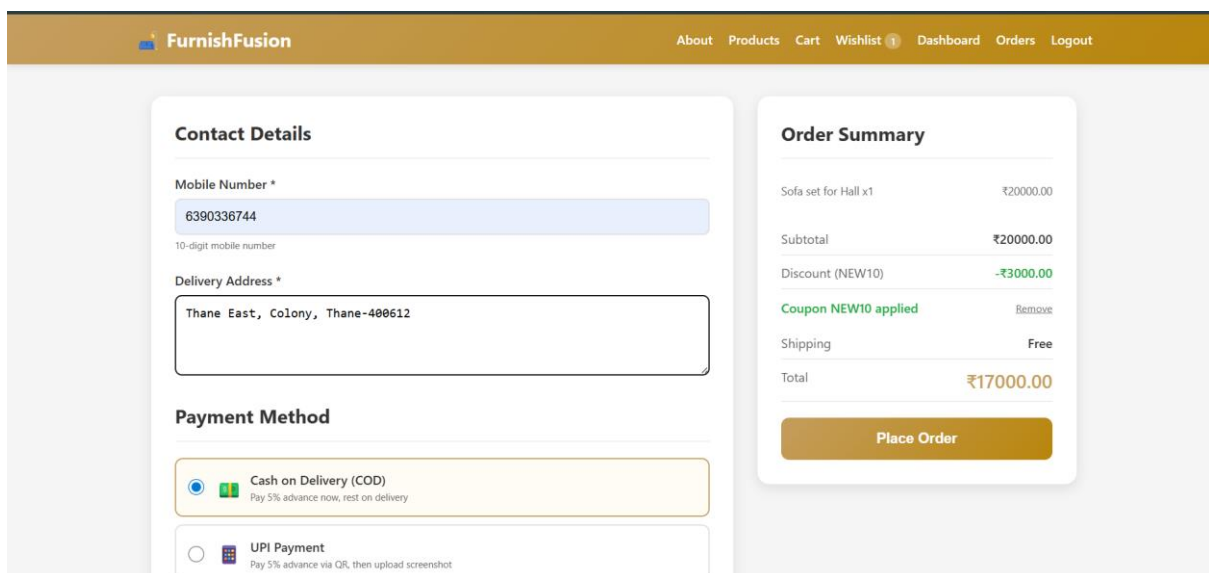
Products Dashboard :



Cart Section to proceed to Checkout :



Order placing Section :



User side Order Tracking :

User Side Order Status Checking

← → ↻ ⚠ Not secure 34.236.134.29:5000/orders ☆ 📄 School

Order #11

Placed on: 2026-02-06T14:38:23

PENDING

₹17000.00

Sofa set for Hall

White coloured Royal look for any Room

Qty: 1 ₹20000.00

Payment Method: UPI Payment

AWAITING_VERIFICATION

✖ Cancel Order

📦 Order Tracking

Order Placed

2026-02-06T14:38:23

Order Accepted

Processing

Shipped

Delivered

← → ↻ ⚠ Not secure 34.236.134.29:5000/orders ☆ 📄

Order #3

Placed on: 2026-01-17 00:27:28.598960

COMPLETED

₹35000.00

Wooden Dining Table

Elegant 6-seater dining table made from premium oak wood.

Qty: 1 ₹35000.00

★ Rate your products

Wooden Dining Table

3

Nice product

Submit

Payment Method: Cash on Delivery

PENDING

📦 Order Tracking

Order Placed

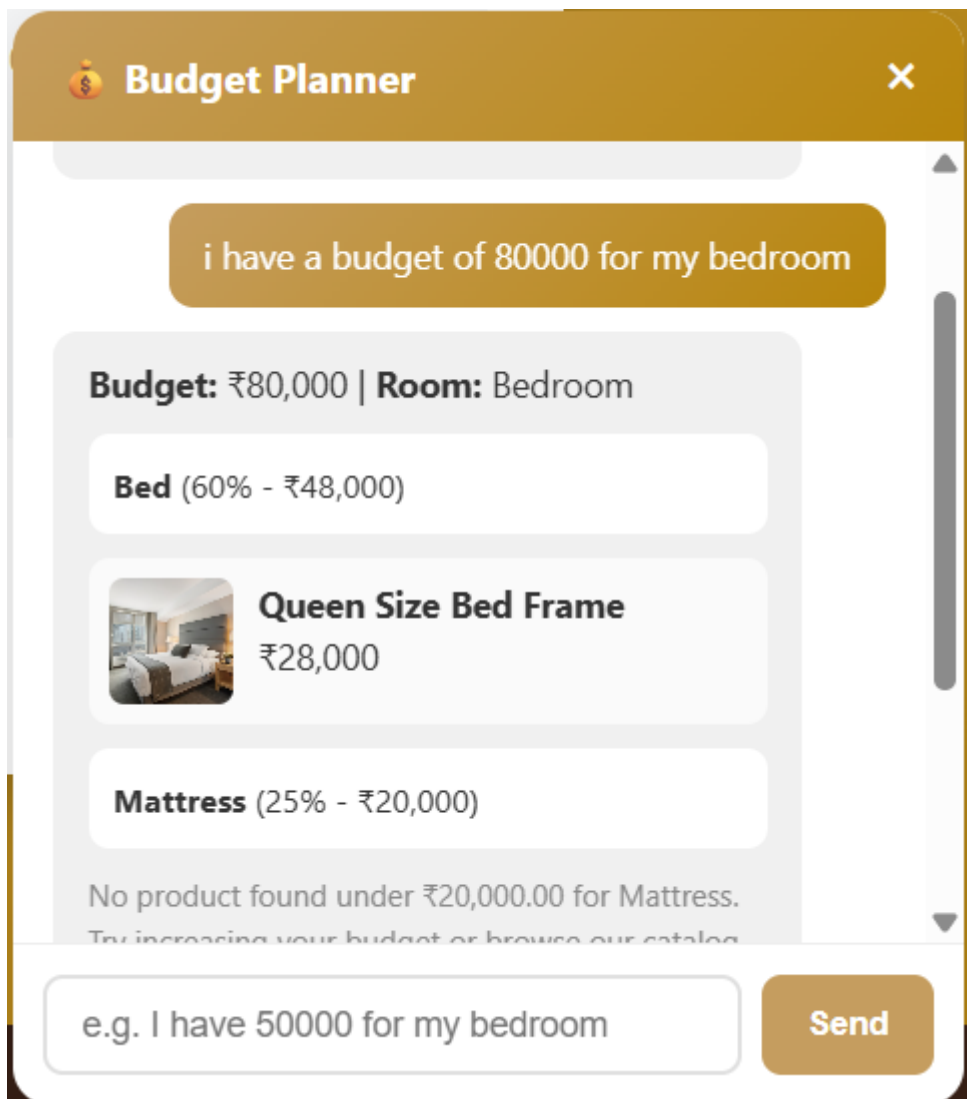
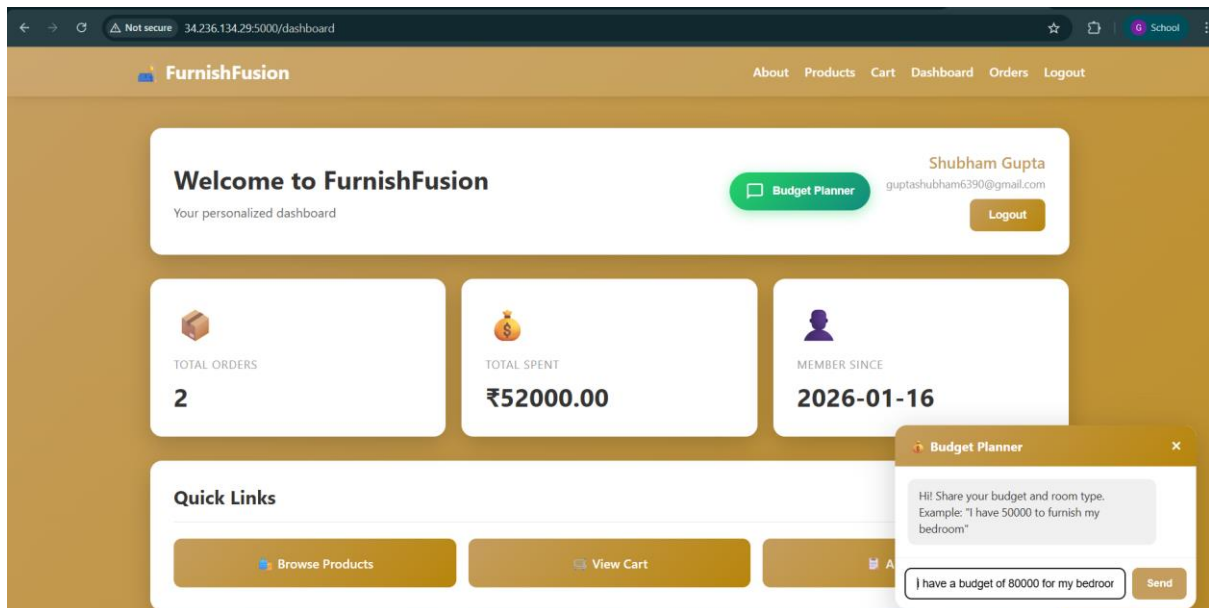
Order Accepted

Processing

Shipped

Delivered

2026-01-17 00:27:28.598960



← → ↻ Not secure 34.236.134.29:5000/login

🔍 ☆ 📄 School

Welcome Back

Login to FurnishFusion

You have been logged out successfully.

User Login

Admin Login

Username

admin

Password

Admin Login

Admin Dashboard – Centrally manage all the tasks like Updating, adding and deleting Products And Reviewing all orders :

← → ↻ Not secure 34.236.134.29:5000/admin/dashboard

🔍 ☆ 📄 School

FurnishFusion Admin Dashboard Products Orders Contact View Site Logout

Welcome, admin!

Admin Dashboard

Welcome, admin!


TOTAL PRODUCTS
10

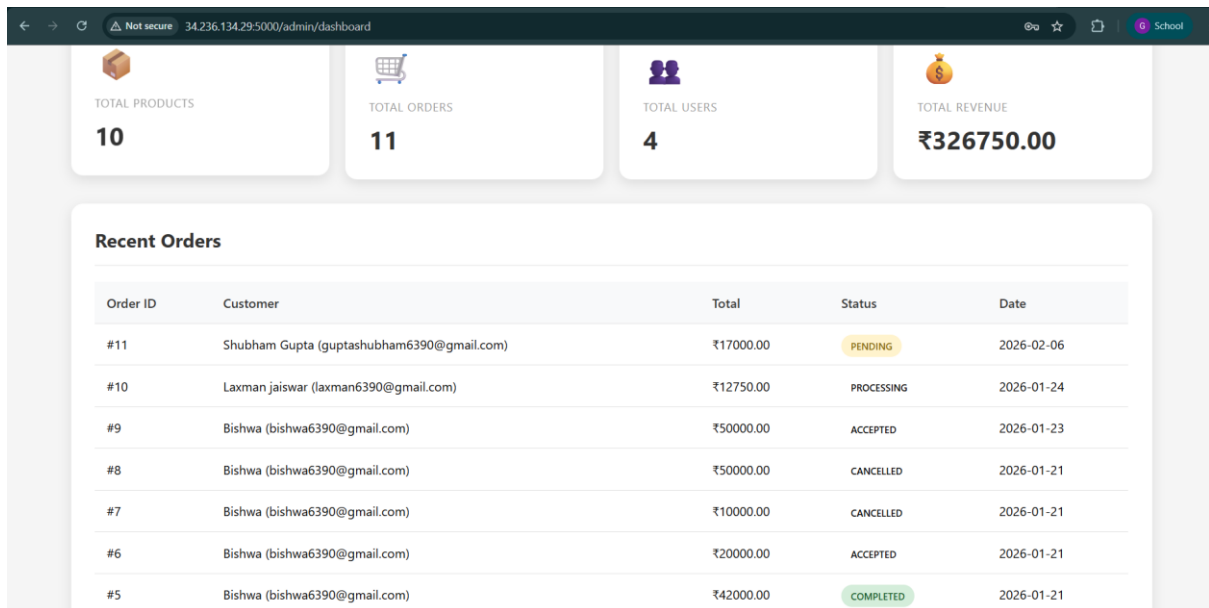

TOTAL ORDERS
11


TOTAL USERS
4

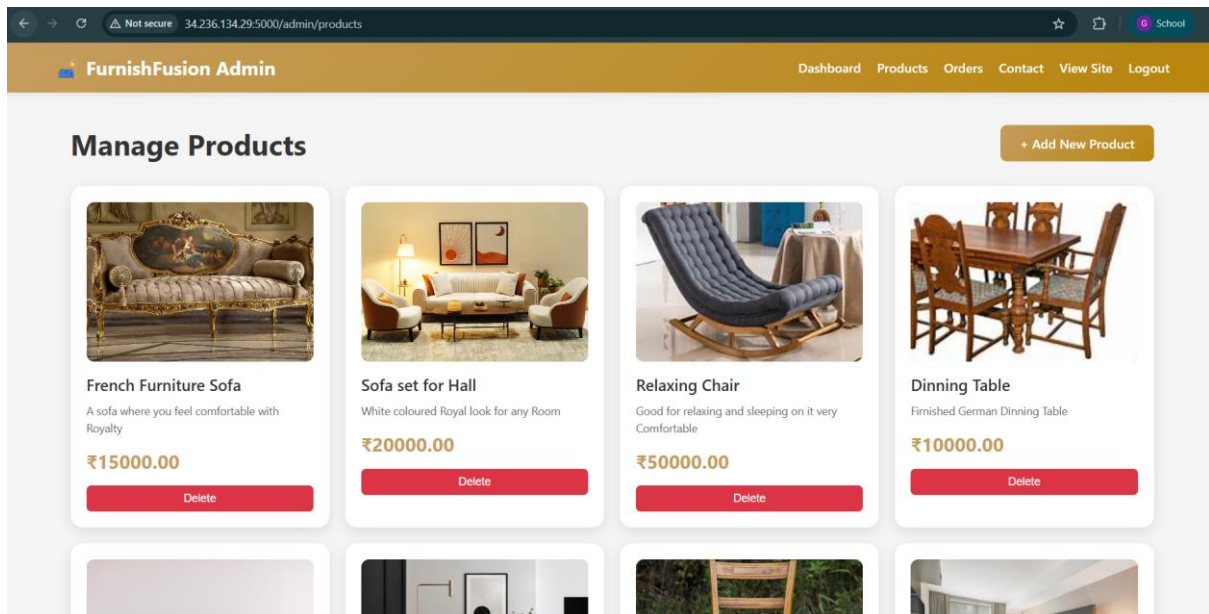

TOTAL REVENUE
₹326750.00

Recent Orders

Order ID	Customer	Total	Status	Date
#11	Shubham Gupta (guptashubham6390@gmail.com)	₹17000.00	PENDING	2026-02-06



Adding and Deleting Products :



Managing Orders :

The screenshot shows the 'Manage Orders' page in the FurnishFusion Admin interface. The page header includes navigation links: Dashboard, Products, Orders, Contact, View Site, and Logout. The main content area displays details for 'Order #11'. The customer is Shubham Gupta (guptashubham6390@gmail.com), and the order was placed on 2026-02-06T14:38:23. The order status is 'PENDING' with a total amount of ₹17000.00. Below this, the payment method is 'UPI' and the status is 'AWAITING_VERIFICATION' with an advance of ₹850.00. A 'View payment screenshot' link is provided. The contact details section shows the mobile number 6390336744 and the address Thane East, Colony, Thane-400612. There are two buttons: 'Accept Order' (green) and 'Cancel Order' (red). The order items section lists 'Sofa set for Hall' with a quantity of 1 and a price of ₹20000.00.

Order #11
Customer: Shubham Gupta (guptashubham6390@gmail.com)
Placed on: 2026-02-06T14:38:23

PENDING ₹17000.00

Payment: UPI | Status: AWAITING_VERIFICATION | Advance: ₹850.00 [View payment screenshot](#)

Contact Details:
Mobile: 6390336744
Address: Thane East, Colony, Thane-400612

[✓ Accept Order](#) [✗ Cancel Order](#)

Order Items:

Sofa set for Hall White coloured Royal look for any Room	Qty: 1 ₹20000.00
---	------------------

Admin side UPI and Coupon code setting :

The screenshot shows the 'UPI QR Code' and 'Coupons' settings page in the FurnishFusion Admin interface. The page header includes navigation links: Dashboard, Products, Orders, Contact, View Site, and Logout. The 'UPI QR Code' section has a title 'UPI QR Code' and a description 'Upload QR for users to pay 5% advance (shown at checkout when they choose UPI)'. There is a QR code image placeholder. To the right, there is a 'Choose File' button and a 'No file chosen' text. Below this, there is a 'Save QR' button. The 'Coupons' section has a title 'Coupons' and a description 'Only active coupons give discounts. Users enter the code at checkout.' There are three input fields: 'Code (e.g. SAVE10)', 'Percent %', and 'Value'. There is an 'Add' button.

UPI QR Code
Upload QR for users to pay 5% advance (shown at checkout when they choose UPI).



Upload QR image
[Choose File](#) No file chosen

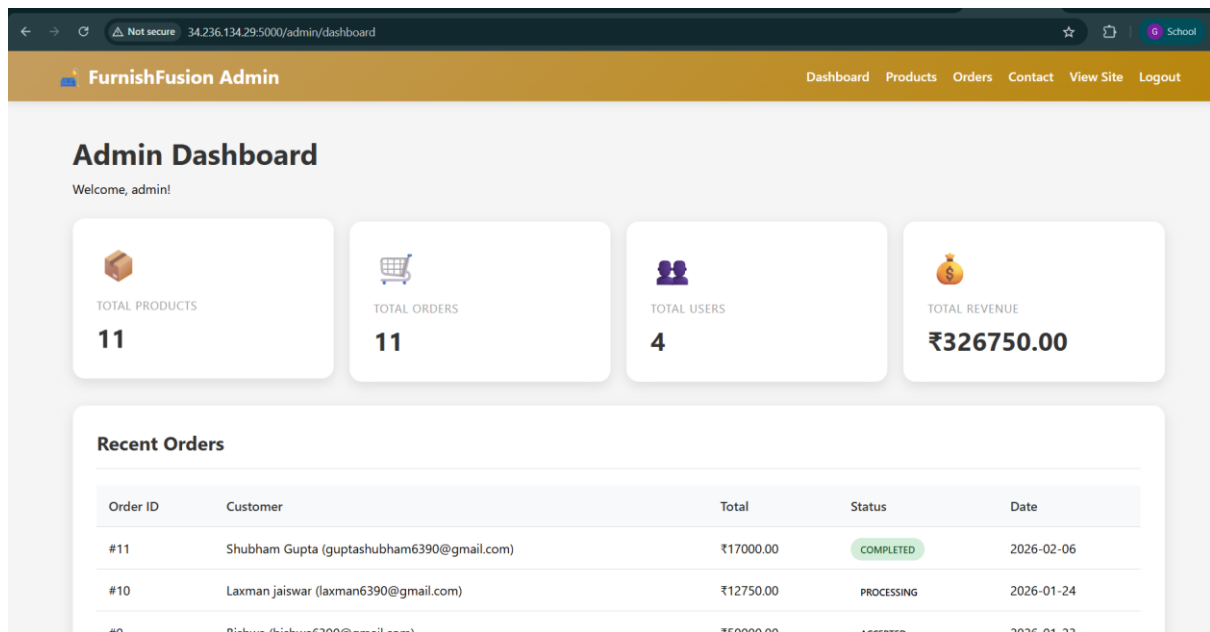
Or image URL

[Save QR](#)

Coupons
Only active coupons give discounts. Users enter the code at checkout.

[Add](#)

Admin can centrally see Total revenue, No. of Users, Total Orders and Total Products and also track orders :



Conclusion

FurnishFusion successfully delivers a complete and practical e-commerce solution for furniture shopping by combining a user friendly interface with a secure and scalable backend architecture. The system allows users to register, explore products, manage their cart, and place orders seamlessly, while the admin panel enables efficient product management, order tracking, and overall system control. By implementing a structured database design and role based access, the platform ensures smooth data handling and reliable performance.

The project also demonstrates strong cloud integration using AWS services such as EC2 for hosting, RDS for database management, and Application Load Balancer for traffic distribution. This two tier architecture improves availability, scalability, and security of the application. Proper IAM role configuration and security group settings further enhance system protection.

Overall, FurnishFusion reflects a well planned and technically sound solution that bridges real world business requirements with modern web development and cloud deployment practices. The project not only fulfills its functional objectives but also showcases practical knowledge of full stack development, database design, and cloud infrastructure management.